

Assignment 2- Case Study (8995-G level Task)

Stage 5 Lab Activities

Refactoring SmartCare into a Layered Architecture

HUMAN ANALYSIS -> REFACTOR -> AI REVIEW -> VERIFY | 1 hour

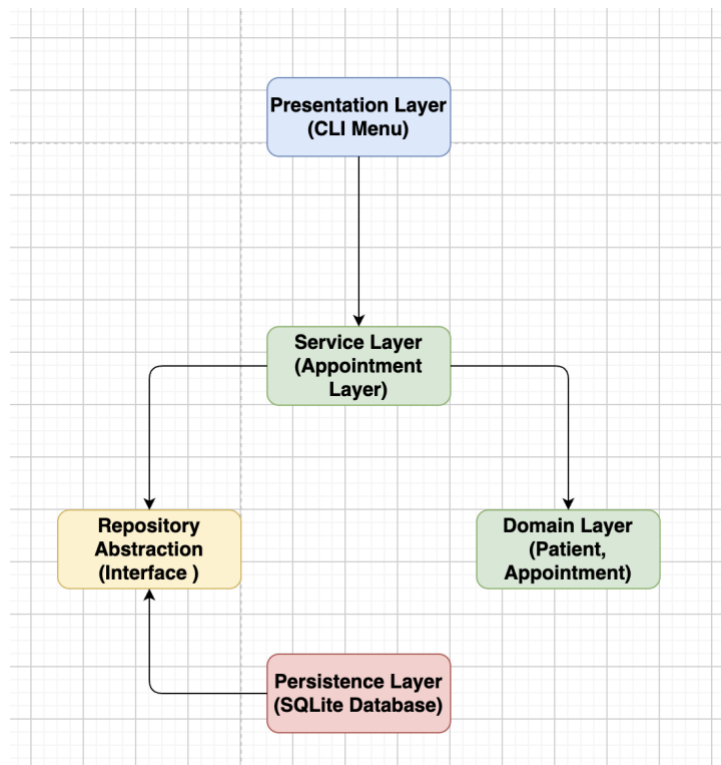
A - Inspect SmartCare v0.4

Identify domain, workflow, data-access and presentation responsibilities currently mixed together.

In the v0.4 code, the main script mixes up input() functions (Presentation), business rules for checking scheduling conflicts (Domain), the step-by-step logic for booking (Workflow), and SQLite INSERT commands (Data-Access) all inside the same file.

B - Propose Architecture

Draw Presentation -> Service -> Domain, with Service using a Repository abstraction and Persistence implementing it.



C - Create Package Structure

Create domain/, services/, repositories/, persistence/ and presentation/ or a justified equivalent.

Done

D - Introduce AppointmentService

Move workflow coordination into a focused service without stealing Appointment domain behaviour.

```
class AppointmentService:
```

```
    def __init__(self, repository):
```

```
        self.repository = repository # Uses the abstraction
```

```
    def book_appointment(self, appointment):
```

```
        # Coordinates the booking workflow
```

```
        self.repository.save(appointment)
```

E - Repository Abstraction

Define a small AppointmentRepository contract using only current use-case needs.

```
class AppointmentRepository:
```

```
    def save(self, appointment):
```

```
        pass # To be implemented by the persistence layer
```

```
    def find_by_id(self, appointment_id):
```

```
        pass # To be implemented by the persistence layer
```

F - AI Architecture Review

Ask AI to review dependency direction, misplaced responsibilities and unnecessary complexity; request simplest justified improvements.

I used the suggested prompt to ask the AI to check my work. The AI pointed out a dependency risk where my service was still relying on the database directly, which is a layer violation. It suggested using the repository abstraction to fix this misplaced responsibility.

G - Refactor and Verify

Apply only justified changes and confirm required behaviour remains unchanged.

I applied the justified changes by moving all the SQL code into the new persistence folder so the service layer no longer contained database logic. I ran the basic manual tests to confirm that booking and cancelling appointments still worked exactly the same way as before.

H - Reflection

Document one AI suggestion accepted, one modified and one rejected/deferred.

During the review, I accepted the AI's suggestion to introduce an AppointmentRepository abstraction because it solved the dependency risk of having the service directly execute SQLite commands. I modified the AI's suggestion to build a massive interface, but instead I restricted it strictly to the current use cases (saving and finding) to keep it simple. Finally, I completely rejected the AI's proposal to split the application into microservices, as that introduces unnecessary complexity and violates the requirement for a small, maintainable system.

Suggested AI prompt

Act as a software architecture reviewer. Review this small SmartCare Python system against separation of concerns, cohesion, coupling and introductory SOLID principles. Identify concrete layer violations and dependency risks. Prefer the simplest refactoring that solves an observed problem. Do not introduce frameworks, microservices or patterns unless current requirements justify them.